Git y GitHub

Control de versiones y trabajo colaborativo en proyectos

Estructuras de Datos II

Problema común al trabajar sin Git

Cuando se trabaja sin control de versiones, suelen aparecer archivos como:

- proyecto_final.cpp
- proyecto_final_corregido.cpp
- proyecto_final_ahora_si.cpp
- proyecto_final_ultimo.cpp
- proyecto_final_ultimo_bueno.cpp

Esto causa problemas porque:

- No sabemos qué cambió.
- No sabemos quién modificó algo.
- Es difícil regresar a una versión anterior.
- Se pueden perder avances importantes.

¿Qué es Git?

Git es un sistema de control de versiones.

Permite guardar el historial de cambios de un proyecto.

Con Git podemos:

- Registrar cambios importantes.
- Volver a versiones anteriores.
- Comparar modificaciones.
- Trabajar por ramas.
- Colaborar con otros desarrolladores.

¿Qué es GitHub?

GitHub es una plataforma en línea para alojar repositorios Git.

Git funciona en la computadora local.
GitHub permite subir el proyecto a Internet.

Con GitHub podemos:

- Guardar una copia remota del proyecto.
- Compartir código.
- Trabajar en equipo.
- Revisar cambios.
- Documentar proyectos.
- Entregar prácticas o tareas.

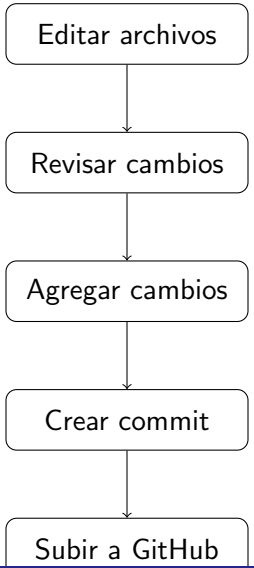
Git vs GitHub

Git	**GitHub**
Programa instalado en la computadora	Plataforma web
Controla versiones del proyecto	Almacena repositorios en línea
Funciona sin Internet	Requiere Internet para sincronizar
Usa comandos como commit, branch, merge	Usa repositorios, issues, pull requests

Conceptos básicos

- **Repositorio:** carpeta del proyecto controlada por Git.
- **Commit:** fotografía del estado del proyecto.
- **Branch:** línea de trabajo independiente.
- **Merge:** unión de cambios de una rama con otra.
- **Remote:** repositorio remoto, por ejemplo en GitHub.
- **Push:** subir cambios al repositorio remoto.
- **Pull:** traer cambios desde el repositorio remoto.
- **Clone:** copiar un repositorio remoto a la computadora.

Flujo general de trabajo



Configurar Git por primera vez

Antes de trabajar con Git, se recomienda configurar el nombre y correo:

```
git config --global user.name "Nombre del estudiante"
git config --global user.email "correo@ejemplo.com"
```

Para revisar la configuración:

```
git config --list
```

Crear un repositorio local

Entramos a la carpeta del proyecto:

```
cd mi_proyecto
```

Inicializamos Git:

```
git init
```

Esto crea una carpeta oculta llamada:

```
.git
```

Esa carpeta guarda el historial del proyecto.

Revisar el estado del proyecto

El comando más usado para revisar cambios es:

```
git status
```

Este comando permite ver:

- Archivos nuevos.
- Archivos modificados.
- Archivos eliminados.
- Archivos listos para commit.

Agregar archivos al área de preparación

Para agregar todos los archivos modificados:

```
git add .
```

Para agregar solo un archivo:

```
git add main.cpp
```

El área de preparación indica qué cambios serán incluidos en el próximo commit.

Crear un commit

Un commit guarda una versión del proyecto:

```
git commit -m "Agrega recorrido preorden"
```

Un buen mensaje de commit debe ser:

- Claro.
- Corto.
- Específico.
- Escrito en presente o infinitivo.

Ejemplos:

- Agrega clase Nodo
- Corrige eliminación de nodo hoja
- Actualiza README

Ver el historial de cambios

Para ver los commits realizados:

```
git log
```

Versión resumida:

```
git log --oneline
```

Cada commit tiene un identificador único, por ejemplo:

```
a3f45c2 Agrega clase ArbolBinario
```

¿Qué debe incluir un commit?

Un commit debe representar un cambio lógico.

Ejemplos correctos:

- Agregar una función.
- Corregir un error.
- Mejorar documentación.
- Crear una práctica nueva.

Ejemplos incorrectos:

- Subir muchos cambios sin relación.
- Usar mensajes como “cambios” o “avance”.
- Hacer commit de archivos temporales.

Estados de un archivo en Git

Un archivo puede estar en diferentes estados:

- **Untracked:** Git todavía no lo sigue.
- **Modified:** fue modificado.
- **Staged:** está listo para commit.
- **Committed:** ya fue guardado en el historial.

Flujo:

$$\text{Modificar archivo} \rightarrow \text{git add} \rightarrow \text{git commit}$$

Conectar con GitHub

Primero se crea un repositorio en GitHub.

Después, desde la terminal:

```
git remote add origin https://github.com/usuario/repositorio.git
```

Para subir por primera vez:

```
git push -u origin main
```

Después normalmente basta con:

```
git push
```

Clonar un repositorio

Clonar significa descargar un repositorio existente.

```
git clone https://github.com/usuario/repositorio.git
```

Esto crea una carpeta con:

- Los archivos del proyecto.
- El historial de Git.
- La conexión al repositorio remoto.

Actualizar el proyecto

Para traer cambios desde GitHub:

```
git pull
```

Este comando se usa cuando:

- Se trabaja desde varias computadoras.
- Otro compañero subió cambios.
- El repositorio remoto tiene cambios nuevos.

Recomendación:

```
git pull
git status
```

Ramas en Git

Una rama permite trabajar sin afectar directamente la versión principal.

Ejemplo:

- **main:** versión estable del proyecto.
- **login:** desarrollo del inicio de sesión.
- **arbol-avl:** práctica de árboles AVL.
- **correccion-error:** arreglo específico.

Las ramas ayudan a experimentar sin romper el proyecto principal.

Comandos básicos para ramas

Crear una rama:

```
git branch nueva-rama
```

Cambiarse a una rama:

```
git checkout nueva-rama
```

Crear y cambiarse al mismo tiempo:

```
git checkout -b nueva-rama
```

Unir una rama con la actual:

```
git merge nueva-rama
```

Conflictos

Un conflicto ocurre cuando Git no puede decidir automáticamente qué cambio conservar.

Ejemplo:

- Dos personas modifican la misma línea.
- Una persona elimina un archivo y otra lo modifica.
- Se mezclan cambios diferentes en la misma sección.

Para resolver un conflicto:

1. Revisar el archivo marcado.
2. Elegir qué código conservar.
3. Guardar el archivo corregido.
4. Hacer `git add`.
5. Crear el commit de resolución.

Ejemplo de conflicto

Git marca un conflicto así:

```
<<<<<<< HEAD
cout << "Version local";
=======
cout << "Version remota";
>>>>>>> rama
```

El programador debe editar el archivo y dejar solo la versión correcta:

```
cout << "Version corregida";
```

Archivo README

El README es la presentación principal del proyecto.

Normalmente se escribe en formato Markdown.

Debe responder preguntas como:

- ¿Qué hace el proyecto?
- ¿Cómo se organiza?
- ¿Cómo se compila o ejecuta?
- ¿Qué herramientas usa?
- ¿Quién lo desarrolló?
- ¿Qué prácticas contiene?

Estructura básica de un README

Ejemplo:

````
# Nombre del proyecto

Descripcion breve del proyecto.

## Organizacion

- 01_Arboles
- 02_Arboles_AVL
- 03_Arboles_B

## Compilacion

```bash
g++ main.cpp -o programa
./programa
```
````

Buenas prácticas para README

Un buen README debe ser:

- Claro.
- Ordenado.
- Fácil de leer.
- Actualizado.
- Útil para alguien que no conoce el proyecto.

No debe ser solo un archivo vacío.

El README ayuda a que el proyecto sea entendible y profesional.

Archivo .gitignore

El archivo `.gitignore` indica qué archivos no deben subirse al repositorio.

Ejemplos de archivos que normalmente se ignoran:

- Ejecutables.
- Archivos temporales.
- Carpetas de compilación.
- Configuraciones locales.
- Archivos generados automáticamente.

Esto mantiene limpio el repositorio.

Ejemplo de .gitignore para C++

```
# Ejecutables
*.exe
*.out

# Archivos objeto
*.o

# Carpetas de compilacion
build/
dist/

# Archivos temporales
*.tmp
*.log

# Configuracion de VS Code
.vscode/
```

Trabajo individual con Git

Para proyectos individuales, el flujo recomendado es:

1. Crear o clonar el repositorio.
2. Trabajar en los archivos.
3. Revisar cambios con `git status`.
4. Agregar cambios con `git add`.
5. Crear commit.
6. Subir a GitHub con `git push`.

Este flujo permite tener evidencia del avance.

Trabajo en equipo con GitHub

Para proyectos en equipo, GitHub permite:

- Compartir un repositorio.
- Crear ramas para cada funcionalidad.
- Revisar cambios antes de unirlos.
- Documentar problemas.
- Coordinar tareas.

La idea es evitar que todos modifiquen directamente la versión principal sin control.

Pull Request

Un Pull Request es una solicitud para integrar cambios.

Sirve para:

- Revisar código antes de unirlo.
- Comentar errores o mejoras.
- Validar que el proyecto siga funcionando.
- Mantener control sobre la rama principal.

En proyectos escolares, también ayuda a ver qué aportó cada integrante.

Issues

Los Issues permiten registrar pendientes, errores o tareas.

Ejemplos:

- Corregir impresión del árbol.
- Agregar recorrido por niveles.
- Revisar eliminación de nodos.
- Mejorar README.
- Agregar capturas de ejecución.

Funcionan como una lista de trabajo dentro del repositorio.

Organización recomendada para proyectos escolares

Ejemplo para una materia:

- `README.md`
- `.gitignore`
- `01_Arboles/`
- `02_Recorridos/`
- `03_AVL/`
- `04_Arboles_B/`
- `docs/`
- `capturas/`

La estructura debe permitir encontrar fácilmente cada práctica.

Ejemplo de flujo completo

```
git clone https://github.com/usuario/proyecto.git
cd proyecto

git status

# Editar archivos del proyecto

git add .
git commit -m "Agrega practica de arboles AVL"
git push
```

Este ciclo se repite durante todo el desarrollo.

Errores comunes

- No hacer commits frecuentemente.
- Usar mensajes poco claros.
- Subir archivos innecesarios.
- No actualizar el README.
- Trabajar siempre en la rama principal.
- No hacer `pull` antes de trabajar.
- Subir código que no compila.

Buenas prácticas

- Hacer commits pequeños y claros.
- Probar el código antes de subirlo.
- Mantener actualizado el README.
- Usar nombres descriptivos para carpetas y archivos.
- No subir archivos generados innecesariamente.
- Revisar `git status` constantemente.
- Usar ramas cuando el cambio sea grande.

Git como evidencia de trabajo

Git no solo guarda código.

También muestra:

- Cuándo se trabajó.
- Qué cambios se hicieron.
- Quién realizó cada cambio.
- Cómo evolucionó el proyecto.

Por eso es útil para proyectos escolares, prácticas, exámenes y trabajos en equipo.

Actividad sugerida en clase

Crear un repositorio para una práctica sencilla.

Pasos:

1. Crear una carpeta llamada `practica-git`.
2. Inicializar Git.
3. Crear un archivo `README.md`.
4. Crear un archivo `main.cpp`.
5. Hacer el primer commit.
6. Crear un repositorio en GitHub.
7. Conectar el repositorio local con GitHub.
8. Subir el proyecto.

Comandos de la actividad

```
mkdir practica-git
cd practica-git

git init

echo "# Practica Git" > README.md

git add .
git commit -m "Crea estructura inicial"

git remote add origin https://github.com/usuario/practica-git.git
git push -u origin main
```

Resumen

- Git permite controlar versiones.
- GitHub permite alojar repositorios en línea.
- Los commits guardan avances importantes.
- El README documenta el proyecto.
- El `.gitignore` evita subir archivos innecesarios.
- Las ramas permiten trabajar sin afectar la versión principal.
- Git es una herramienta esencial para proyectos de programación.

Cierre

Git y GitHub no solo sirven para subir código.

Sirven para trabajar de manera ordenada, profesional y colaborativa.

En proyectos escolares, ayudan a demostrar el avance real del trabajo y a mantener un historial claro de cada práctica.